

Sistema de Gestión de Asignaturas

Instituto “Ctrl+Alt+Clase”

Nombre:	
Focus Group:	Modelando Bases de datos.
Carrera:	Analista Programador – Ingeniería en Informática.
Docente:	Giancarlo Favio Jaque Ojeda.
Habilidades a practicar	Identificación de entidades, atributos, claves, relaciones y restricciones.
Resultado esperado	Modelo Entidad-Relación Extendido (M.E.R.E.) o modelo lógico-relacional desarrollado en Oracle SQL Developer Data Modeler, con entidades, atributos, claves, relaciones y restricciones correctamente representadas según el caso planteado.

¡IMPORTANTE! En esta guía no se entrega el diagrama listo. El desafío es leer el caso, reconocer la información relevante y construir el modelo que permita representar el funcionamiento académico del instituto.

INSTRUCCIONES GENERALES PARA EL DESARROLLO DEL EJERCICIO

Antes de comenzar el desarrollo del caso práctico, lea completamente el enunciado e identifique con claridad las entidades que forman parte del sistema, los datos que debe almacenar cada una, las relaciones existentes entre ellas y las restricciones que deben respetarse para que la base de datos funcione correctamente.

Para el desarrollo de la actividad, considere las siguientes instrucciones generales:

- 1) Antes de construir el modelo, analice el problema e identifique las entidades principales, sus atributos, claves primarias, claves foráneas y relaciones.
- 2) Las tablas deben ser diseñadas con nombres claros y representativos, respetando la estructura lógica del caso y evitando duplicar información innecesariamente.
Cada tabla debe contar con una clave primaria que permita identificar de manera única sus registros.
- 3) Los atributos deben utilizar tipos de datos adecuados según la información que almacenan, considerando textos, números, fechas, códigos e identificadores.
- 4) Las relaciones entre las entidades deben reflejar correctamente las reglas del caso, por ejemplo.
- 5) No se recomienda utilizar elementos no revisados en clases ni modificar la estructura del caso sin justificación. El objetivo es construir un modelo claro, funcional y coherente con los requerimientos planteados.
- 6) Antes de finalizar, revise que el modelo permita representar correctamente la información del caso y que no existan relaciones sueltas, datos repetidos innecesariamente o tablas sin propósito claro dentro del caso.

1. Contexto del caso.

El Instituto Ctrl+Alt+Clase es una institución de formación técnica que partió hace algunos años con **cursos** pequeños, una **sede** arrendada y un equipo **docente** bastante reducido. Al comienzo todo se podía manejar con una planilla compartida, un par de **correos** y la memoria heroica de la **coordinación académica**. Sin embargo, el instituto creció rápido: abrió nuevas **jornadas**, sumó más **secciones** por **asignatura**, incorporó docentes **especialistas** y empezó a recibir **estudiantes** de distintas **comunas**.

Hoy el instituto ofrece asignaturas en **modalidad diurna y vespertina**, distribuye clases en distintas **salas**, organiza cursos por secciones y necesita saber con claridad qué docente está a cargo de cada asignatura. Además, como las **clases** se desarrollan en diferentes espacios físicos, la **coordinación** debe conocer no solo la sala asignada, sino también el **piso** en que está ubicada y la sede a la que pertenece. Esto es especialmente importante cuando se realizan cambios de sala, **evaluaciones presenciales**, actividades recuperativas o ajustes de **horario** a mitad de semestre.

Actualmente, la información académica se encuentra repartida en varias planillas. Una contiene los datos de docentes, otra los datos de estudiantes, otra las asignaturas del **semestre** y otra, bastante desactualizada, las salas disponibles. Esto ha provocado errores frecuentes: correos institucionales mal escritos, asignaturas con secciones duplicadas, salas sin capacidad registrada y estudiantes inscritos en cursos que después son difíciles de rastrear.

También existen problemas al momento de cruzar información. Por ejemplo, la coordinación puede saber el **nombre de una asignatura**, pero no siempre tiene claro qué docente la imparte, qué estudiantes la cursan o qué sala se está utilizando. Del mismo modo, administración puede tener registrada una sala, pero no siempre puede identificar rápidamente en qué piso está ni a qué sede pertenece.

El problema no es solo guardar datos y cruzarlos, sino poder confiar en ellos. Cuando una estudiante pregunta dónde tiene clases, cuando un docente necesita confirmar qué asignatura imparte o cuando administración debe revisar la capacidad disponible de las salas, el sistema debe entregar respuestas consistentes. Por eso, la dirección decidió dejar atrás las planillas sueltas y diseñar una base de datos académica que ordene la información central del instituto.

Para resolver esto, el instituto necesita que el nuevo sistema permita organizar la información de manera relacional. Es decir, cada dato debe quedar guardado en el lugar que corresponde y las conexiones entre datos deben estar claramente representadas. La meta es que el sistema ayude a responder preguntas reales del funcionamiento diario, sin depender de búsquedas manuales ni de registros repetidos.

2. Requerimiento general.

Debes diseñar un Modelo Entidad-Relación Extendido (M.E.R.E.) para el Sistema de Gestión de Asignaturas del Instituto Ctrl+Alt+Clase. A partir del caso descrito, tendrás que identificar las entidades principales, reconocer sus atributos, definir claves primarias y establecer las relaciones necesarias para representar el funcionamiento académico del instituto.

El modelo debe ser desarrollado en Oracle SQL Developer Data Modeler, respetando las reglas del caso y representando de forma clara las entidades, atributos, claves, cardinalidades y restricciones que correspondan.

3. Información del sistema que debes modelar.

En el funcionamiento del instituto aparecen varios elementos que deberás reconocer y transformar en entidades del modelo. La coordinación trabaja con docentes que imparten clases, estudiantes que cursan asignaturas, asignaturas que se dictan por sección y jornada, salas donde se realizan las clases, pisos donde se ubican esas salas y sedes que agrupan los pisos disponibles. No se entregan

estas estructuras como tabla lista, porque parte del ejercicio consiste en que identifiques qué representa cada elemento dentro del sistema y cómo debería convertirse en una tabla.

Los datos personales e institucionales de docentes y estudiantes deben permitir distinguir a cada persona mediante su **RUN**. Para cada **docente** se necesita guardar **primer nombre, segundo nombre, primer apellido, segundo apellido, fecha de nacimiento y correo institucional**. Para cada estudiante se debe considerar la misma lógica de identificación y contacto: **RUN, primer nombre, segundo nombre, primer apellido, segundo apellido, fecha de nacimiento y correo institucional**. Estos datos deben ayudarte a decidir cuáles atributos son obligatorios, cuáles identifican de manera única y cuáles podrían requerir restricciones para evitar duplicidad.

Cada **asignatura** debe quedar registrada con un **identificador interno, un nombre, un código, una sección y una jornada**. Por ejemplo, una misma asignatura podría dictarse en más de una sección o en jornadas distintas, por lo que el diseño debe permitir diferenciar esos casos sin confundirlos. Las **salas también requieren un identificador interno, un código reconocible para la comunidad académica y una capacidad máxima de estudiantes**. En cuanto a la infraestructura, cada **piso necesita un identificador y un número de piso**, mientras que cada **sede debe registrar un identificador, su nombre y su dirección**.

4. Relaciones y limitaciones del sistema.

El sistema debe representar que las asignaturas son cursadas por estudiantes y que los docentes imparten asignaturas. También debe reflejar que una asignatura utiliza una sala para el desarrollo de sus clases. A nivel de infraestructura, las salas se ubican en pisos y los pisos pertenecen a una sede. Estas relaciones deben quedar modeladas de forma que el instituto pueda cruzar información sin duplicarla, por ejemplo, consultar una asignatura y conocer sus estudiantes, su docente, la sala asignada, el piso de esa sala y la sede correspondiente.

Debes poner atención a las cardinalidades. En un semestre, un estudiante puede cursar varias asignaturas y una asignatura puede tener varios estudiantes, por lo que esa relación no debería

resolverse como un simple dato escrito dentro de una columna. Para efectos de este caso, considera que un docente puede impartir varias asignaturas y que una asignatura puede ser impartida por más de un docente cuando existan distintas secciones o jornadas. Tu diseño debe justificar si esta relación requiere una tabla intermedia y evitar que se repitan datos innecesariamente.

La ubicación física tiene una dependencia más directa: una sala debe estar asociada a un piso existente, y un piso debe pertenecer a una sede existente. No debería ser posible registrar una sala en un piso que no exista ni un piso asociado a una sede inexistente. Además, la **capacidad máxima de una sala** debe ser un valor coherente, los correos institucionales no deberían repetirse dentro del sistema y los **códigos de sala** o asignatura deberían mantenerse claros para evitar confusiones operativas.

¡OJO! Antes de escribir SQL, identifica qué relaciones pueden resolverse con una clave foránea directa y cuáles necesitan una tabla intermedia. Esa decisión es parte central del ejercicio.

5. Tips y restricciones técnicas

- Utiliza nombres de tablas y columnas claros, consistentes y relacionados con el caso académico.
- Usa tipos de datos compatibles con la naturaleza de cada atributo, como VARCHAR para textos, NUMERIC para identificadores o capacidades, CHAR para valores breves y DATE para fechas.
- Define claves primarias para identificar de manera única cada registro principal.
- Declara claves foráneas donde corresponda para proteger la integridad de las relaciones.
- Evita registros duplicados en correos institucionales, códigos de sala y códigos de asignatura cuando el motor de base de datos lo permita.
- Mantén el modelo coherente con el caso: no agregues entidades, atributos o relaciones que no estén justificados por la situación descrita.